

Code Walkthrough: Weather Agent with Tool-Calling

Line-by-line notes on `weather_tool.py` — how Gemini decides to call your Python function

This script is a step up from a plain weather-printer: instead of you fetching the weather and stuffing it into a prompt yourself, you hand Gemini a **tool** (a function it's allowed to call) and let the model decide when it needs weather data to answer the user's question. This pattern is called **function calling** or **tool use**, and it's the basic building block of AI agents.

1. Imports

```
import requests as req
from google import genai
from dotenv import load_dotenv
import os
import json
```

Same as before — **requests** for HTTP calls, **genai** for the Gemini SDK, **dotenv/os** for reading API keys from a `.env` file. New here is **json**, Python's built-in module for converting between Python objects (dicts, lists) and JSON text. It's needed because when we send the weather result back to Gemini, it has to be serialized as a JSON string, not a raw Python dict.

2. Loading environment variables

```
load_dotenv()
```

Reads your `.env` file and loads `WEATHER_API_KEY` and `GEMINI_API_KEY` into the environment, exactly as covered in the previous script — this must run before any `os.getenv()` calls.

3. The `get_weather` function

```
def get_weather(city):
    url_weather = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={os.getenv('WEATHER_API_KEY')}"
    res_weather = req.get(url_weather)
    weather_data = res_weather.json()
    temp = weather_data["main"]["temp"]
    description = weather_data["weather"][0]["description"]
    return {"city": city, "temp": temp, "description": description}
```

This is the exact same weather-fetching logic as before, just wrapped in a reusable function that takes a **city** string and **returns** a small Python dictionary instead of printing anything. Wrapping it as a function is what makes it possible to expose it to Gemini as a callable tool in the next steps.

4. Creating the Gemini client

```
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))
```

Same as before — this authenticates your connection to the Gemini API using your key from `.env`.

5. Describing the tool to Gemini

```

weather_tool = {
    "type": "function",
    "name": "get_weather",
    "description": "Gets the weather conditions for a given city.",
    "parameters": {
        "type": "object",
        "properties": {
            "city": {
                "type": "string",
                "description": "The city name, e.g. San Francisco",
            },
        },
        "required": ["city"],
    },
}

```

The model can't see your Python code, so you describe the tool to it in a structured JSON schema. This tells Gemini: there's a function called **get_weather**, here's what it does in plain English (used by the model to decide *when* to call it), and it takes one required argument called **city**, which is a string. This schema format (type/name/description/parameters) is a common convention across most LLM APIs that support tool use.

6. Mapping tool names to real functions

```

available_functions = {
    "get_weather": get_weather,
}

```

The model only ever gives you back the *name* of the function it wants to call, as text (e.g. "get_weather"). This dictionary is a lookup table that maps that name string to the actual Python function object, so your code can execute it dynamically later with `available_functions[name](...)`.

7. Getting the user's question

```

user_input = input("Ask me anything: ")
previous_id = None

```

Takes free-form input from the user — it could be anything, not just a weather question. **previous_id** is initialized to `None` and will be used to chain multiple turns of the same conversation together (see next step).

8. The main loop: calling Gemini

```

while True:
    interaction = client.interactions.create(
        model="gemini-3.5-flash",
        input=user_input,
        tools=[weather_tool],
        previous_interaction_id=previous_id,
    )

```

This runs forever until explicitly broken out of (step 10). Each time through, it sends **user_input** to Gemini along with the list of available **tools**. **previous_interaction_id** links this call to the prior one, so Gemini

keeps context of the conversation rather than treating each call as a fresh, memoryless request — similar to how a chat thread keeps history.

9. Checking if the model wants to call a function

```
function_results = []
for step in interaction.steps:
    if step.type == "function_call":
        result = available_functions[step.name](**step.arguments)
        print(f"Called {step.name}({step.arguments}) → {result}")
        function_results.append({
            "type": "function_result",
            "name": step.name,
            "call_id": step.id,
            "result": [{"type": "text", "text": json.dumps(result)}],
        })
```

Gemini's response can contain multiple **steps** — text, or a request to call a tool. This loop scans through them, and whenever it finds a **function_call** step, it looks up the real function via **available_functions** and calls it, unpacking the model's chosen arguments with ****step.arguments** (e.g. if the model decided `city="Tokyo"`, this becomes `get_weather(city="Tokyo")`). The result dictionary is then serialized to a JSON string with **json.dumps()** and packaged into the format Gemini expects for function results, tagged with the same **call_id** so the model knows which call this result answers.

10. Deciding whether to loop again or stop

```
if not function_results:
    break

user_input = function_results
previous_id = interaction.id
```

If the model didn't request any function calls this round, that means it already has everything it needs and produced a final answer — so the loop **breaks**. Otherwise, the results of the function call(s) are fed back in as the next 'input' to Gemini (as if the tool were replying to it), and **previous_id** is updated so the next call continues the same conversation thread. Gemini then gets another turn to either call more tools or produce a final text answer.

Note: This request → function_call → execute → feed result back → request again cycle is the core loop behind virtually all tool-using AI agents, regardless of provider.

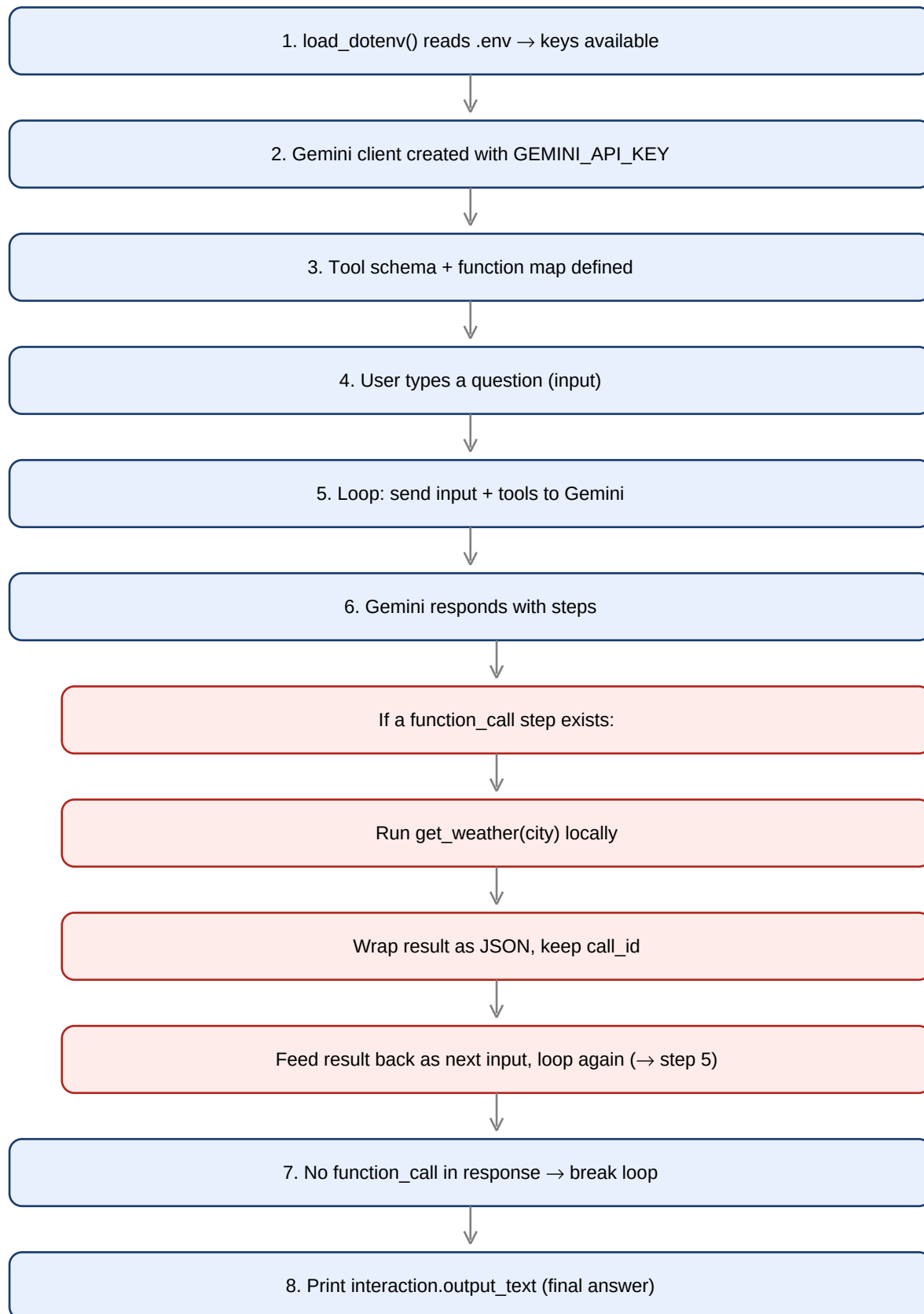
11. Printing the final answer

```
print(interaction.output_text)
```

Once the loop breaks, **interaction** holds the model's last response — the one that had no further function calls, i.e. its final natural-language answer. That's printed to the terminal.

Sequence of Execution

Below is the step-by-step order the program actually runs in, including the loop and its branch:



Key Idea

The difference from the first script: instead of you deciding to fetch weather every time, you give the model the *option* to fetch it — and only when it decides the question actually needs live data. If you had asked "what's 2+2" instead of a weather question, the model would likely skip calling the tool entirely and answer directly on the first loop iteration.